Session 4
General Specifications

M2104 Object-Oriented Design and Modelling - Prepare the specifications for a program

2021 Edition

Starting an Object-based Design

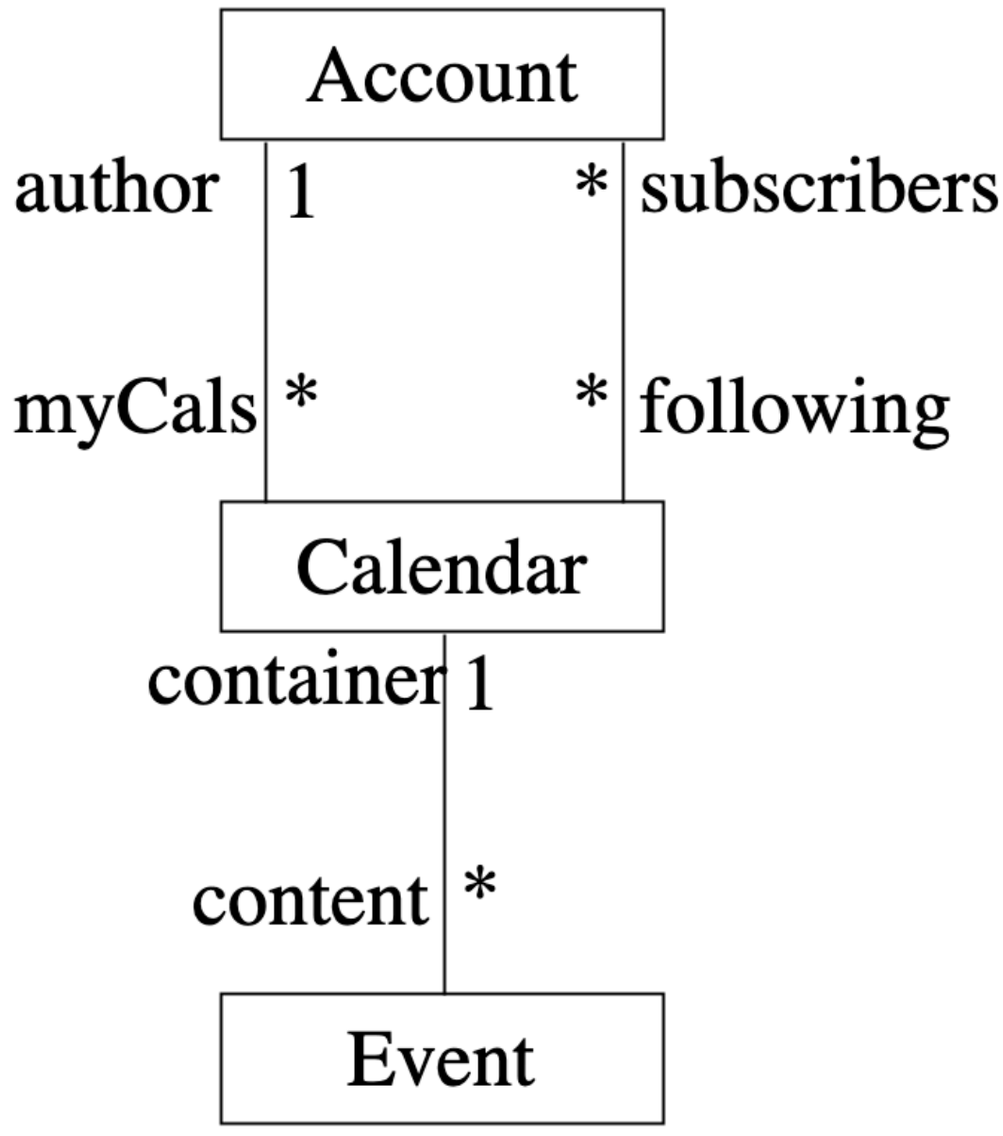
- **Needs** have been defined (i. e., actors, events, use cases)
- Next is a **first analysis** of the basic structuring elements of the software: **objects** (and thus classes)
- **What entities** of the program appear? These will be the different kinds of objects (and thus, again, classes)
- **What class will be responsible** for each part of the software? What services will it provide? (and thus what methods)
- In order to do so, **what data** will it require? (and therefore, what attributes)
- **What entities need what other entities** to perform their functions? This will be the basis for the relationships between classes (associations, use links…)

The general specification class diagram

- ***Design starts with two main questions:***
  - What classes? What relationships between those classes?

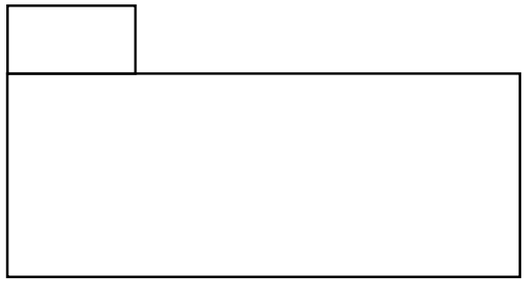
- The “simplest” version of the class diagram (general diagram, skeleton):
  - rectangle boxes with a name (for each class),
  - relationships between classes: associations (roles, multiplicities, optionally navigabilities), use links…
  - nothing more.

Example (S3Ex3)



Addition: packages in UML

- Group elements of the same kind

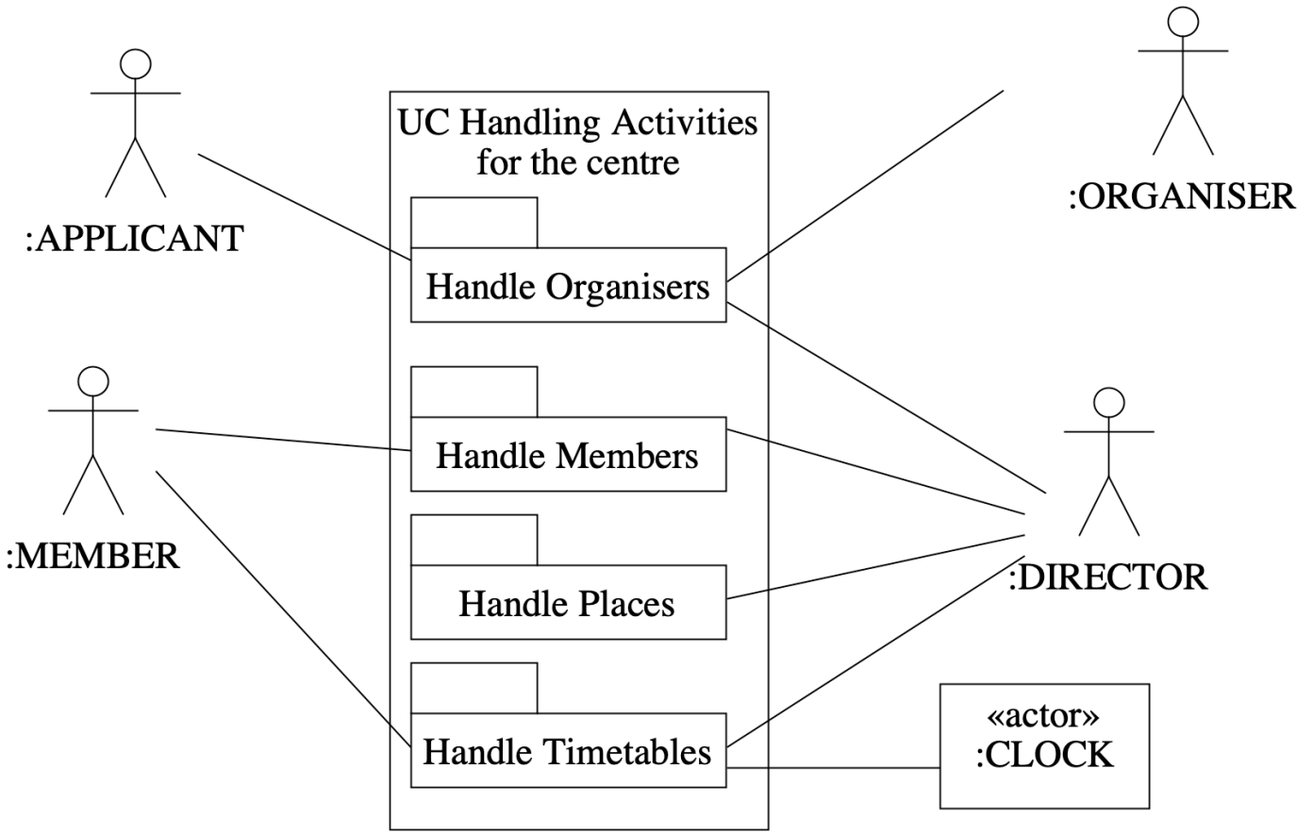


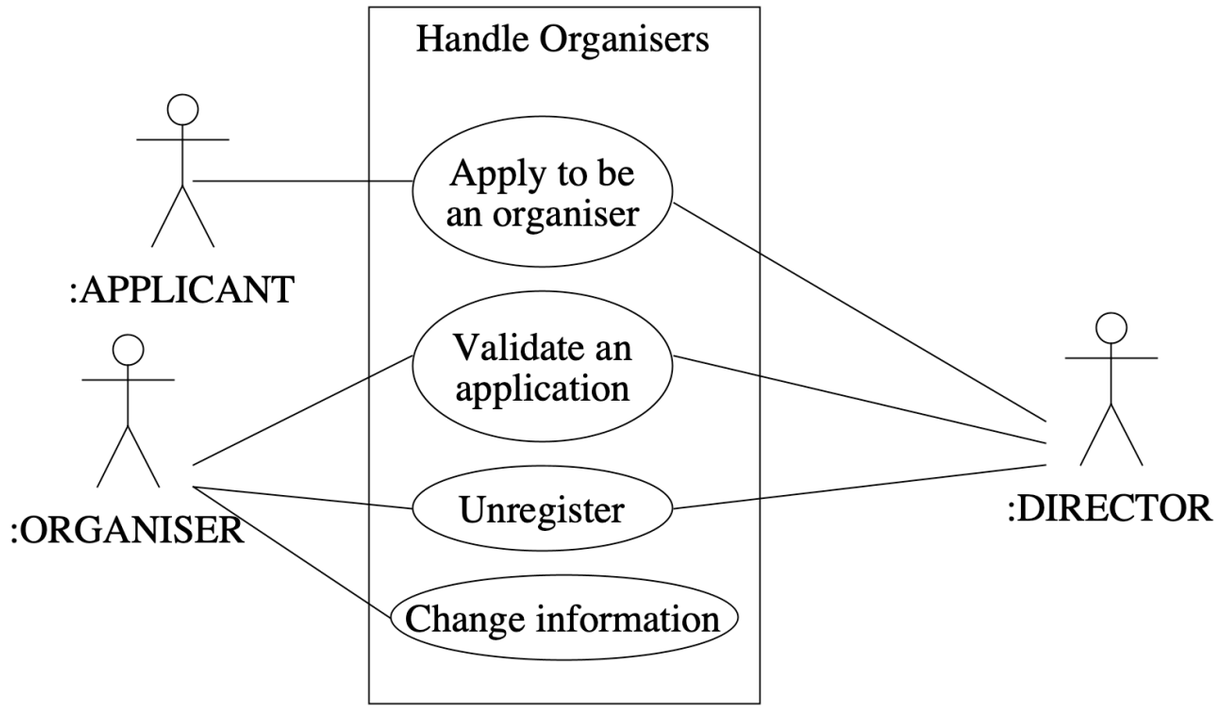
- No *package diagrams* this year, but we can use packages:
  - in the use case diagram,
  - in the class diagram

Example: the activity monitor

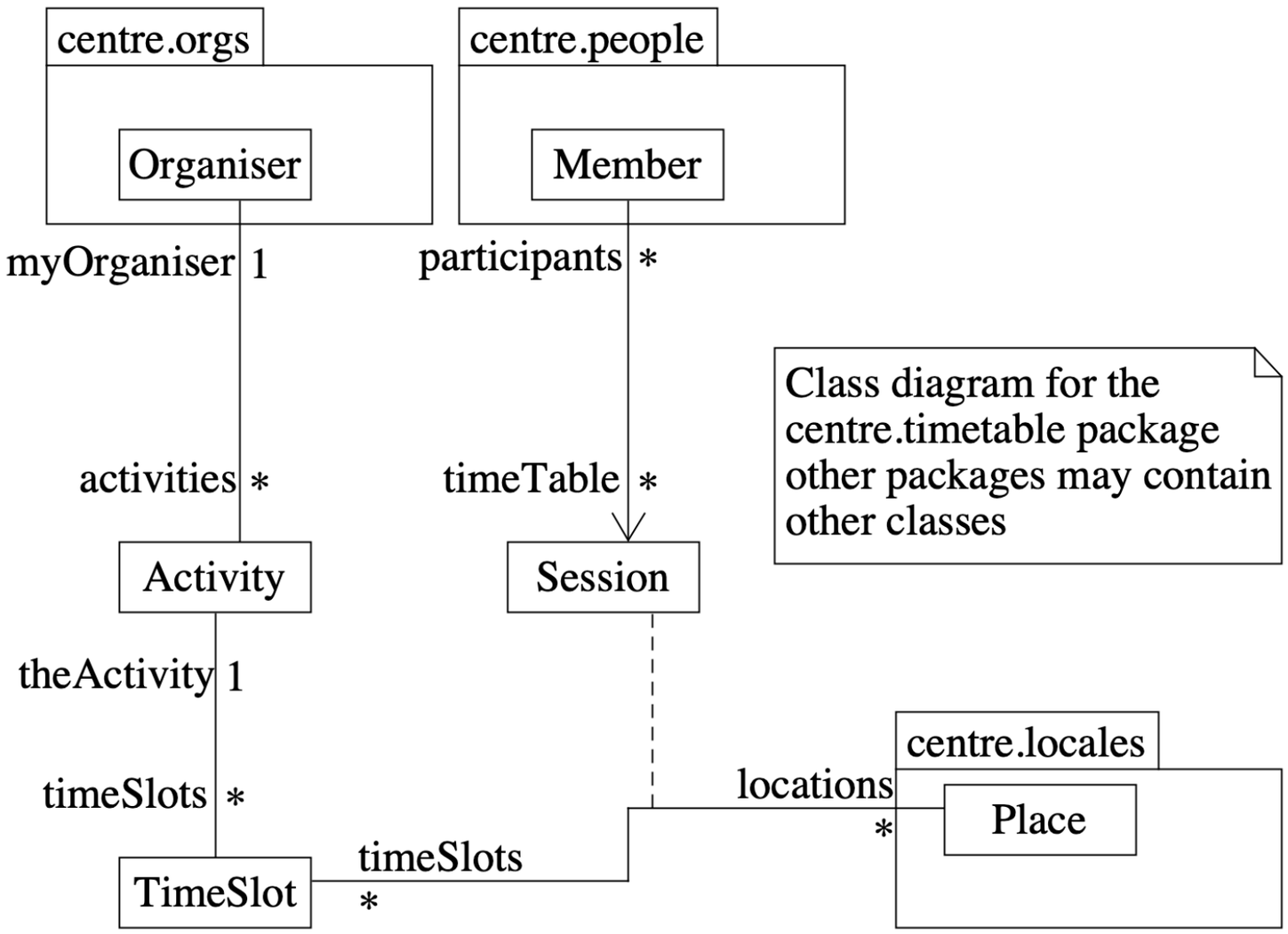
- Theme: some software for an animation centre used to handle activities
  - for many members
  - with many organisers (clubs, institutions…)
  - in different places,
  - on different time slots…
- The full specification would require time (and space), packages allow to focus on specific elements

UC diagram w/ packages





Class diagram w/ packages



Exercise 3: Case Study

Theme	The theme is **Captain's Log** app. A single user, the Captain, has writing permissions for that log with a password, and can save entries with a date, time, title, and specific categories such as *mission start*, *mission report*, *unforeseen event*, *personal*, &c. All other person has writing permissions for the log (no connexion necessary) and may consult all entries from any category except *personal*.
Not required	Modification/deletion of log entries.
Required	Characterise the software, list the external actors, the external and result events, produce a use case diagram, then list the entities that should appear in the software and their responsibilities before producing a general specification class diagram. Grouping classes and use cases in packages should not be necessary for this text.

Exercise 4 : Case Study

Theme	The theme is a website handling **citizen complaints** at the city level. Each citizen may create an account by sending their personal information and a proof of residence, the municipal administration will the validate that account or not depending on the information received. Once the account is created, a citizen may send a complain in the form of a text describing the problem and the problem's category: roads, cleanliness, antisocial behaviour, parking, &c. This complain is marked with the status *to be treated*. The municipal administration may respond with some text and changing the status to *urgent*, *long-term* or *closed* (or keeping the *to be treated* status). Finally, the administration can send the complain to appropriate civil services: municipal technical crews or the police department, each have dedicated accounts. Those services may then also respond to a complain and update its status. While the complain's status is not *closes*, the person that has created it can respond to a response of the administration or services by some text.
Not required	Date handling, reminders, editing previously entered texts.
Required	Characterise the software, list the external actors, the external and result events, produce a use case diagram, then list the entities that should appear in the software and their responsibilities before producing a general specification class diagram. It is up to you to decide if grouping classes and/or use cases in packages is appropriate or not.